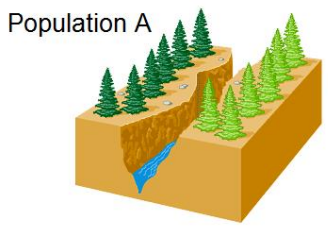


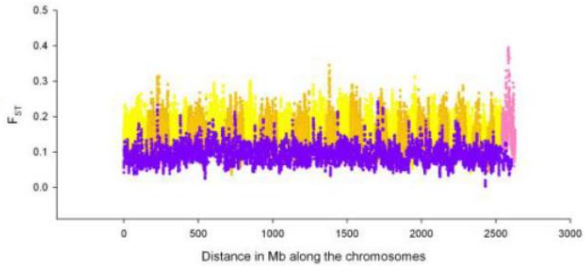
PPP – POPGEN PIPELINE PLATFORM + SNAKEMAKE

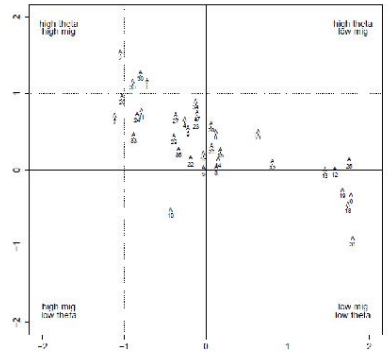
Arun Sethuraman & Trevor Mugoya

San Diego State University


POPGEN PIPELINE PLATFORM (PPP)







Data Preparation

- VCF
- Reference Genome
- Identify populations with STRUCTURE
- F_{st} scan

Multiple Sequence Alignment

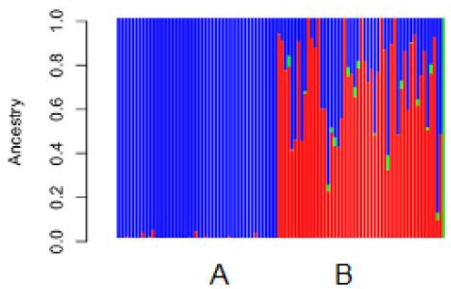
Data Analyses

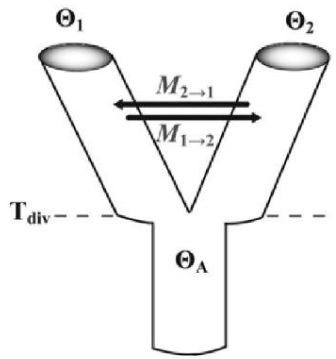
- Demographic analyses IMa2p, mixture model

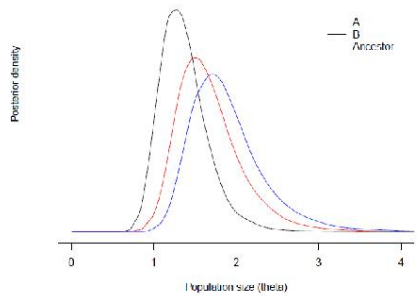
Run-time, post hoc output files

Data Visualization

- Parameter histograms
- Analyses of selection, migration







PPP Core Functions
(VCF-based file Functions)

`vcf_filter`
- Comprehensive VCF filter
- Creates: VCF or BED files

`vcf_split`
- Split VCF using BED/STAT file
- Creates: Directory of VCF files

`vcf_calc`
- Basic Statistic Calculator
- Creates: STAT files

`vcf_phase`
- Phase VCFs using Beagle/SHAPEIT
- Creates: Phased VCF files

`informative_loci_filter`
- Locus-based filter
- Creates: BED files

`vcf_four_gamete`
- Locate regions that pass the FGT
- Creates: FGT-compatible VCF files

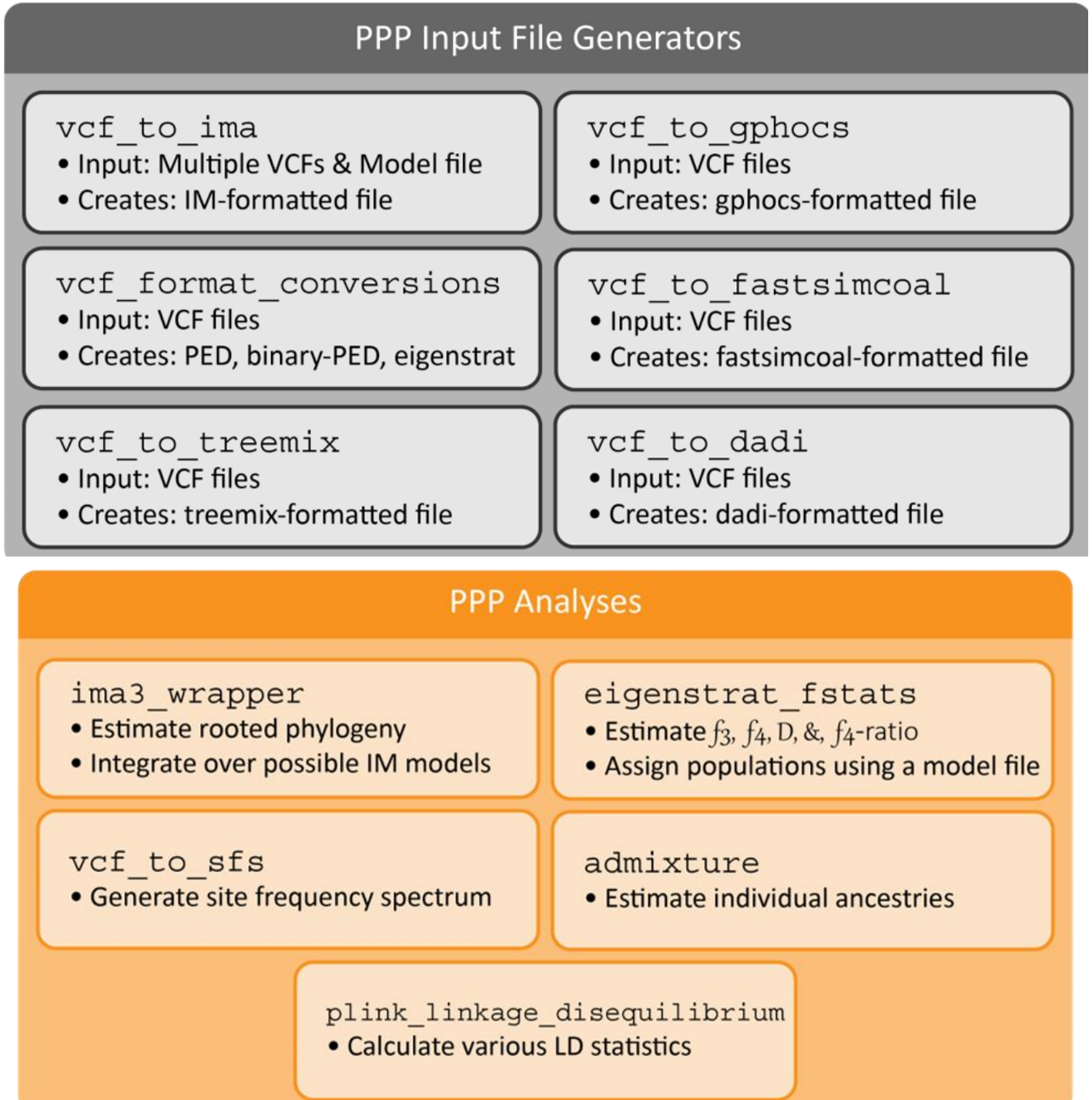
PPP Utilities

`bed_utilities`
- Various BED utilities
- Creates: BED files

`vcf_utilities`
- Various VCF utilities
- Creates: VCF files

`vcf_bed_to_seq`
- Get sequences from VCFs & BEDs
- Creates: Sequences

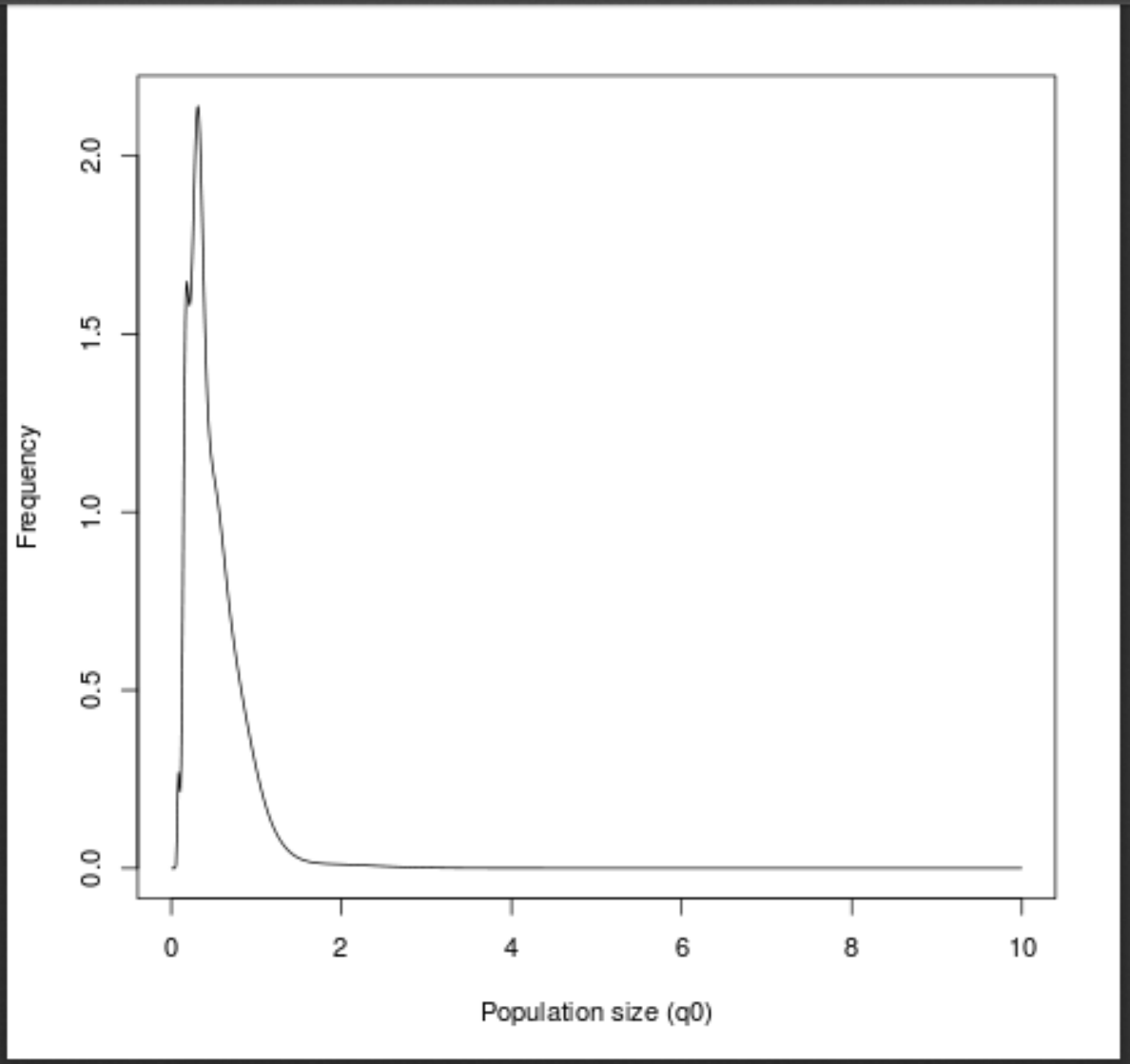
`stat_sampler`
- Random & Uniform Sampling
- Creates: STAT files


PPP Input File Generators
vcf_to_ima
• Input: Multiple VCFs & Model file
• Creates: IM-formatted file
vcf_to_gphocs
• Input: VCF files
• Creates: gphocs-formatted file
vcf_format_conversions
• Input: VCF files
• Creates: PED, binary-PED, eigenstrat
vcf_to_fastsimcoal
• Input: VCF files
• Creates: fastsimcoal-formatted file
vcf_to_treemix
• Input: VCF files
• Creates: treemix-formatted file
vcf_to_dadi
• Input: VCF files
• Creates: dadi-formatted file
PPP Analyses
ima3_wrapper
• Estimate rooted phylogeny
• Integrate over possible IM models
eigenstrat_fstats
• Estimate f_3, f_4, D, &, f_4-ratio
• Assign populations using a model file
vcf_to_sfs
• Generate site frequency spectrum
admixture
• Estimate individual ancestries
plink_linkage_disequilibrium
• Calculate various LD statistics

Tools

search tools

Get Data
Lift-Over
Text Manipulation
Filter and Sort
Join, Subtract and Group
Convert Formats
Extract Features
Fetch Sequences
Fetch Alignments
Statistics
Graph/Display Data
Run IMa2p
Run IMa2p IMa2p Run
Plot histograms Plot histograms
MyTools



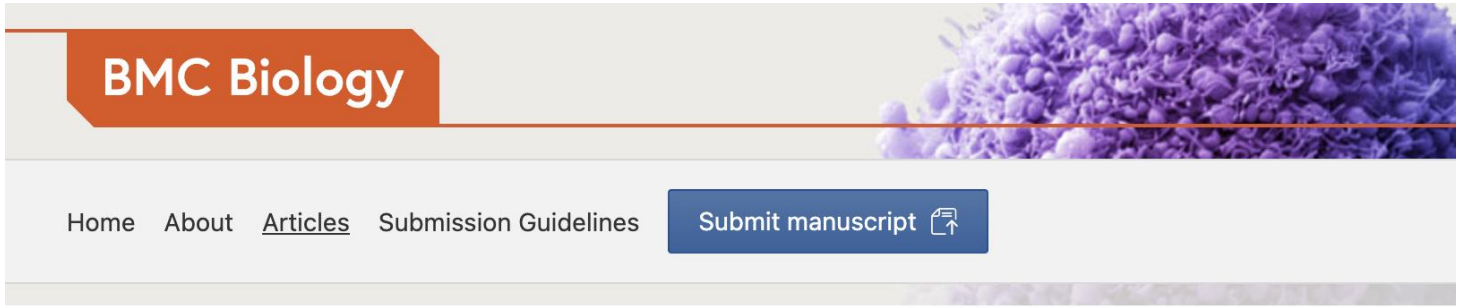
History

search datasets

Unnamed history
30 shown, 148 deleted
9.0 MB

178: Plot histograms on data 174
View data
177: q and m Histograms
176: t0 Histogram
175: Screen output during run
174: Output file produced by IMa2p
173: Screen output during run
172: Output file produced by IMa2p
171: Plot histograms on data 167
170: q and m

Example Workflow – Li et al., 2023



Research article | Open Access | Published: 19 June 2023

Global patterns of genomic and phenotypic variation in the invasive harlequin ladybird

Hongran Li, Yan Peng, Yansong Wang, Bryce Summerhays, Xiaohan Shu, Yumary Vasquez, Hannah Vansant, Christy Grenier, Nicolette Gonzalez, Khyati Kansagra, Ryan Cartmill, Edison Ryoiti Sujii, Ling Meng, Xuguo Zhou, Gábor L. Lövei, John J. Obrycki, Arun Sethuraman ✉ & Baoping Li ✉

BMC Biology **21**, Article number: 141 (2023) | Cite this article



Snakemake Technical Workshop

ConGen 2026.

Introduction.

- A tool for managing complex data analysis workflows
- Based on Python
- Reproducibly automate repetitive tasks

Why Do We Need Workflow Management?

- Managing multiple analysis steps
- Ensuring reproducibility
- Handling dependencies between tasks
- Saves time without recomputing completed tasks (re-entrancy)

Basic Concepts

- Rules - the building blocks of workflows
- Input files - what goes in
- Output files - what comes out
- Shell commands - what actually happens

Installation.

```
conda create -n snakemake -c bioconda snakemake
```

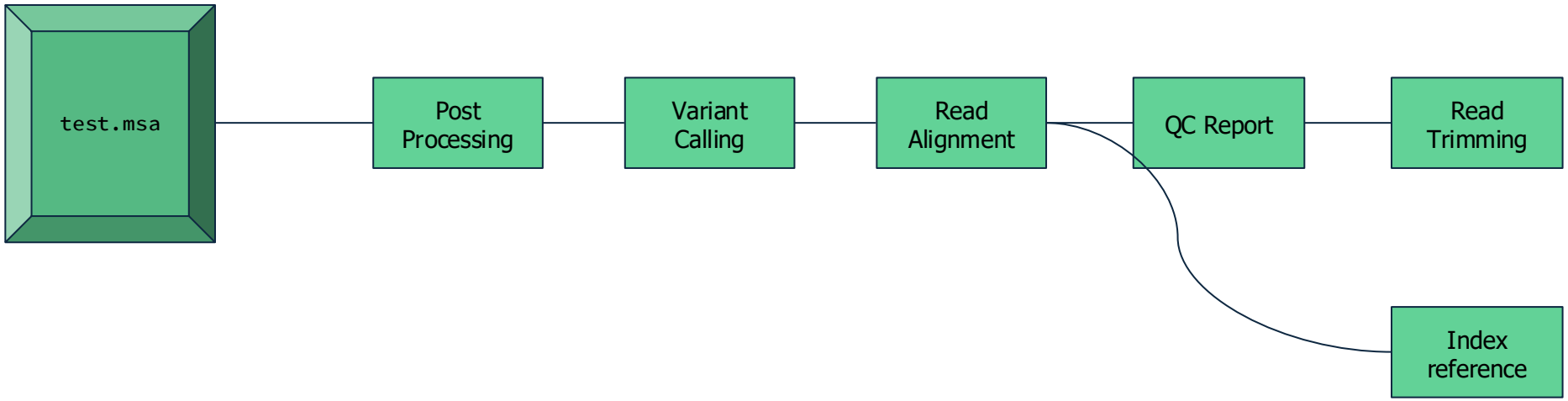


The one rule to rule them all.

- The rule all is a special rule that defines the “end” result of the pipeline.
- Typically this is the last file or set of files that your pipeline will create.
- This rule allow snakemake to work “***backwards***” to generate a sequence of steps that will eventually lead to the file(s) specified in rule all.

```
rule all:
    input:
        ppp_outputs,
```

All roads lead to all (An example DAG)



Example Multiple Sequence Alignment Workflow



Rule Structure

```
rule myrule:
    input:
        "path/to/inputfile",
        "path/to/other/inputfile",
    output:
        "path/to/outputfile",
        "path/to/another/outputfile",
    shell:
        "somecommand {input} {output}"
```

https://snakemake.readthedocs.io/en/stable/snakefiles/rules.html



Rule Structure (Explicit Declaration)

```
rule myrule:
    input:
        in1="path/to/inputfile",
        in2="path/to/other/inputfile",
    output:
        out1="path/to/outputfile",
        out2="path/to/another/outputfile",
    shell:
        "somecommand {input.in1} {input.in1} {output.out1} {output.out1}"
```

https://snakemake.readthedocs.io/en/stable/snakefiles/rules.html

Rule Structure

```
rule myrule:
    input:
        "path/to/inputfile",
        "path/to/other/inputfile",
    output:
        "path/to/outputfile",
        "path/to/another/outputfile",
    shell:
        "somecommand {input} {output}"
```

The `input` directive allows the user to declare one or multiple input files that are subsequently passed on to a shell command or python script.

https://snakemake.readthedocs.io/en/stable/snakefiles/rules.html

Rule Structure

```
rule myrule:
    input:
        "path/to/inputfile",
        "path/to/other/inputfile",
    output:
        "path/to/outputfile",
        "path/to/another/outputfile",
    shell:
        "somecommand {input} {output}"
```

The `output` directive allows the user to declare one or multiple outputs that are generated by the shell command, script or notebook.

One can define absolute or relative paths to desired output files for the rule. File names can be hardcoded or generalized with Snakemake **wildcards*.**

https://snakemake.readthedocs.io/en/stable/snakefiles/rules.html

Rule Structure

```
rule myrule:
    input:
        "path/to/inputfile",
        "path/to/other/inputfile",
    output:
        "path/to/outputfile",
        "path/to/another/outputfile",
    shell:
        "somecommand {input} {output}"
```

The `shell` directive allows the user to declare one or multiple input files that are subsequently passed on to a shell command or python script.

https://snakemake.readthedocs.io/en/stable/snakefiles/rules.html

Managing Results With Output Functions (`common.smk`)

```
def ppp_outputs(wildcards):
        outputs = []
        outputs.extend(
                expand(
                [

    "results/{organism}/popgen.ipynb",

                ],

        organism=config["organism"],
        )
        )
    return outputs
```

One can define python functions to manage input or outputs within a snakemake workflow. The syntax is python however snakemake adds a few features that allow for pipeline flexibility.

Wildcards are one such feature and act as snakemake variables that allow value substitution within declared strings.

Output functions allow us to define all desired outputs to be generated by the workflow as a list that snakemake unpacks to generate it's DAG.

https://snakemake.readthedocs.io/en/stable/snakefiles/rules.html

Demo: